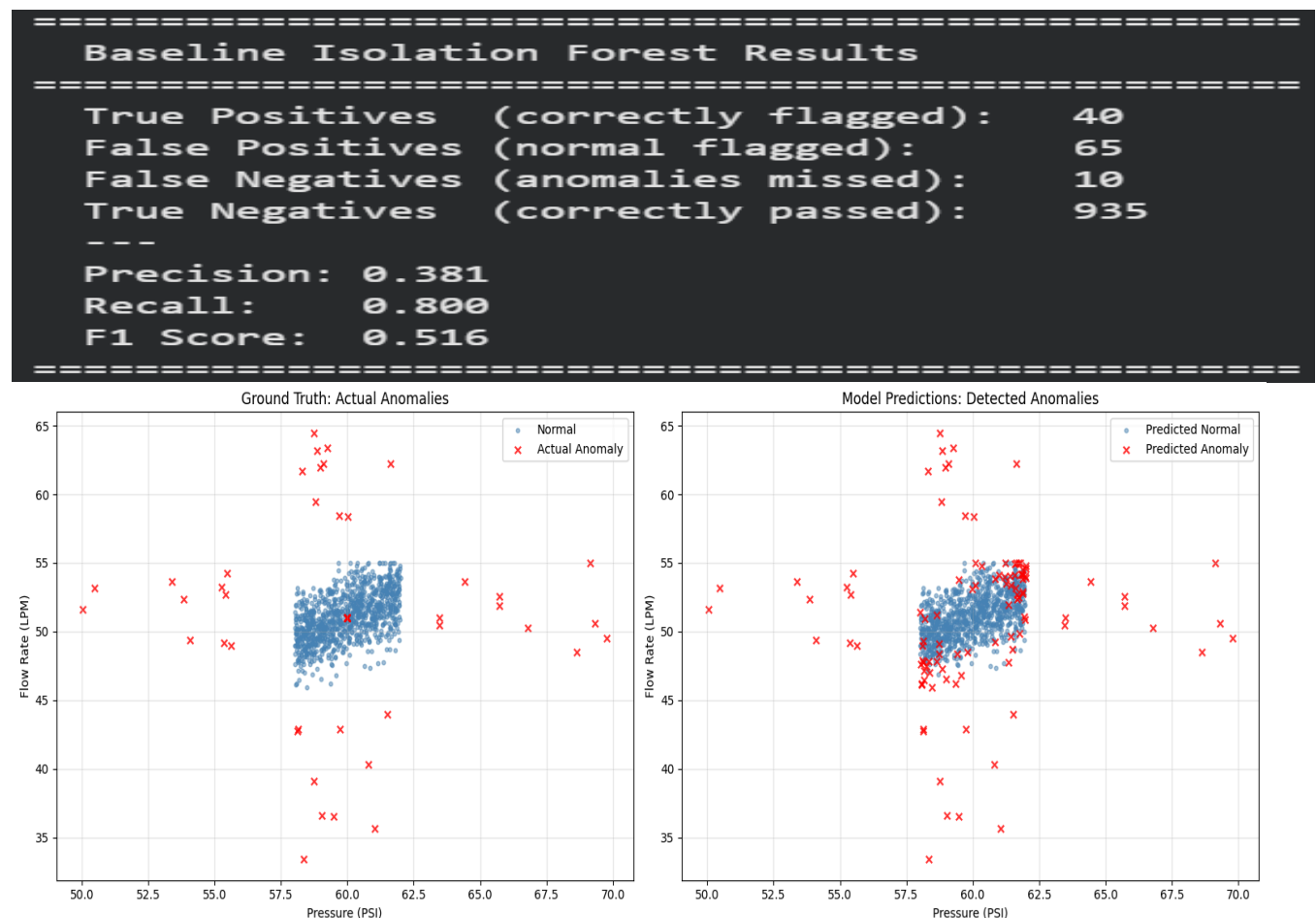


Overview: This week will be focusing on training an AI model that misses certain rules that our defenses cannot pick up on, so anything that looks out of the ordinary we will train it to send out alerts based off of that. We will be using the Isolation Forest algorithm and comparing it to another algorithm called Local Outlier Factor. The point of implementing AI into our system is that it can detect abnormalities that are hard to catch when it comes to ruled based conditions.

Phase 1: Isolation Forest Tuning

- When it comes to machine learning, what values you are putting in matters. I was given a Google colab notebook that does the learning, but the values are what matter, understanding them matter even more. I say this because we have to hit three main goals to have an accurate model for a watering system, **Precision, Recall, and F1 Score**. High Precision = less false alarms on readings, Recall = High recall missed few real anomalies vice versa, F1 Score = Combination of both Precision and Recall, if one value is high and another is low the F1 Score would be rated depending on those 2 values. F1 Scores range from 0 to 1, 1.0 is perfect and 0 is low accuracy.
- I ran a test by changing the model's contamination rate from 5% to 10%, this tells the model to read more anomalies within the given data. Here were the results.



This all looks like random dots but let me explain, the left chart shows accurate anomalies detected, anything outside of the isolated blue dots (normal data) are considered abnormal. The right chart shows What the model itself detected when it comes to anomalies which is pretty accurate. The precision is low meaning more false alarms and a recall of 0.8 shows the accuracy of detecting anomalies but normal data can slip through and cause a false alarm. We get an average F1 score of 0.5 which is a 50%

Phase 2: Running Hyperparameter Experiments

- In this experiment I will take the 3 main parameters (contamination, n_neighbors, max_samples) and combine them at the end to give myself the best training model for anomaly detection. This will include adjusting the values of these parameters and finding the fastest and most accurate results. I will be testing 4 different values to see which is most optimal

Contamination (Percentage of data that will appear anomalous)

Contamination	Detected	Precision	Recall	F1
0.01	11	1.000	0.220	0.361
0.03	32	1.000	0.640	0.780
0.05	53	0.736	0.780	0.757
0.10	105	0.381	0.800	0.516
0.20	210	0.190	0.800	0.308

Best contamination by F1 score: 0.03 (F1 = 0.780)

N_estimators (Higher values give more opinions and stable results)

n_estimators	Precision	Recall	F1	Train Time
50	0.698	0.740	0.718	0.425s
100	0.736	0.780	0.757	0.463s
200	0.717	0.760	0.738	1.077s
500	0.717	0.760	0.738	3.644s

Best n_estimators by F1 score: 100 (F1 = 0.757)

Max_samples (How much data each tree sees)

max_samples	Precision	Recall	F1
auto	0.736	0.780	0.757
64	0.585	0.620	0.602
128	0.698	0.740	0.718
256	0.736	0.780	0.757

Best max_samples by F1 score: auto (F1 = 0.757)

BEST RESULTS:

```

DEFAULT MODEL:

=====
Default Isolation Forest Results
=====
True Positives (correctly flagged):      11
False Positives (normal flagged):        0
False Negatives (anomalies missed):      39
True Negatives (correctly passed):      1000
---
Precision: 1.000
Recall:    0.220
F1 Score:  0.361
=====

YOUR BEST MODEL:

=====
Your Best Configuration Results
=====
True Positives (correctly flagged):      39
False Positives (normal flagged):        14
False Negatives (anomalies missed):      11
True Negatives (correctly passed):      986
---
Precision: 0.736
Recall:    0.780
F1 Score:  0.757
=====

Metric      | Default | Best | Change
-----
precision   | 1.000   | 0.736 | -0.264
recall      | 0.220   | 0.780 | + 0.560
f1          | 0.361   | 0.757 | + 0.397

```

Looking at the metrics from our default results going to our best results there is a huge difference, losing 26% on precision is fine because of the tradeoff with gaining 56% on recalls and 39% on f1 scores which will give us more stable results for our anomaly detection.

Phase 3: Comparing Isolation Forest to Local Outlier Factor

- This comparison will tell us which algorithm is better for our current situation of running the best watering system with high accuracy, Isolation Forest detects anomalies by deviating them from normal data by isolating them, now we will use Local Outlier Factor which tells us how dense is the neighborhood around this point compared to its neighbors.
 - Main parameter: `n_neighbors` controls how many nearby points it looks at when calculating density. More neighbors = smoother estimates but less sensitive to small clusters of anomalies.
 - This experiment will be the same as IF, I will be tuning LOF values to find out which gives the best results.

N_NEIGHBORS (controls nearby points when calculating density)

n_neighbors	Precision	Recall	F1	Time
10	0.755	0.800	0.777	0.022s
20	0.755	0.800	0.777	0.015s
50	0.755	0.800	0.777	0.028s

Best n_neighbors by F1: 10 (F1 = 0.777)

IF Vs LOF (Keep in mind that LOF has a better F1 than IF, but IF would be the better overall winner for anomaly detection due to the fact it's able to score new messages one at a time.)

=====			
HEAD-TO-HEAD: Isolation Forest vs. Local Outlier Factor			
=====			
Metric		IF (Best)	LOF (Best)

Precision		0.736	0.755
Recall		0.780	0.800
F1 Score		0.757	0.777
Training Time		0.705s	0.022s
Can score new data one at a time?		Yes	No
Needs full dataset present?		No	Yes
Overall winner by F1: Local Outlier Factor			

For the Grand Marina the best model to train would be IF, the main reason is for real-time anomaly detection. With IF you can allow new messages to be scored without needing all of the dataset present, LOF might have faster and more accurate detections but it needs the whole dataset so it's not valid in this case. IF is also a lot easier to integrate into an existing subscriber, it's only downside is it might give out slightly more false alarms compared to a LOF, but that is manageable with a good security team.

Phase 4: One-Class SVM

- This model was optional but I wanted to see the results I could get, One-Class SVM draws a boundary around normal data in feature space. Points outside the boundary are anomalies. This model is great when it comes to smaller data giving high accuracy but it is not able to handle large datasets properly.

IF vs LOF vs SVM

One-Class SVM:

```
=====
One-Class SVM Results
=====
True Positives  (correctly flagged):   33
False Positives (normal flagged):      21
False Negatives (anomalies missed):    17
True Negatives  (correctly passed):    979
---
Precision: 0.611
Recall:     0.660
F1 Score:   0.635
=====
Training time: 0.009s
```

THREE-WAY COMPARISON				
Metric	IF	LOF	SVM	
Precision	0.736	0.755	0.611	
Recall	0.780	0.800	0.660	
F1 Score	0.757	0.777	0.635	
Training Time	0.359s	0.017s	0.009s	
Real-time scoring?	Yes	No	Yes	
Needs scaling?	No	No	Yes	

Looking at our results SVM gives us a decently low precision, recall, and f1 score but it does support scaling, in our case this wouldn't be valid. Since SVM only supports smaller data it wouldn't be able to function without our water system setup because of different pressure readings coming in at a rate of 2-5 seconds, Therefore IF is still the best option.

Importing AI Model & Attack Simulation

- After tuning and testing the three AI models I ended up importing the IF joblib file into my vs code so I can use it locally on my machine to pick up anomalies. I will then be adding a few more lines of code so that the AI model (IF) is being used and detecting different types of readings to separate normal data from anomalies.
- I will then be running an anomaly injector to see if the terminal will pick up on it since I have not implemented the detection into my splunk dashboard yet.
 - I will not be displaying code here as it will make the document look messy, everything will be listed within the github with instructions to run it yourself.

Test 1: Anomaly Injection

- After configuring my scripts to read the correct trained model file I am ready to run the test, I will do the same process as before. Open 5 terminals (mqtt broker, subscriber, publisher, anomaly injector, attack simulator)

NORMAL READINGS: As we can see the score is around 100% accuracy.

```
[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 49.1 LPM | Seq: 1
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: Normal (score: 0.090)
Stats: 1 accepted, 0 rejected (1 total)

[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 51.45 LPM | Seq: 2
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: Normal (score: 0.114)
Stats: 2 accepted, 0 rejected (2 total)

[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 47.84 LPM | Seq: 3
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: Normal (score: 0.099)
Stats: 3 accepted, 0 rejected (3 total)
```

ANOMALY INJECTION: We can see that the model works perfectly after injecting readings with higher/lower pressure readings and it also pops up on our subscriber with -1 values meaning anomaly.

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-08-AI-Anomaly-Detection-Dashboard>python anomaly_injector.py

=====
      A N O M A L Y   I N J E C T O R
=====
Target: Grand Marina Hotel
Mode:   Subtle anomalies with VALID signatures
Goal:   Test AI anomaly detection
=====

[INFO] Simulating a second device (Device-002) with valid credentials
[INFO] but subtly abnormal sensor readings.
[INFO] Rule-based checks will PASS. The AI model should flag them.
[INFO] Uses device-001 certs for mTLS (cert authenticates TLS, not payload)

C:\Users\nsain\Documents\hydroficient-project\Week-08-AI-Anomaly-Detection-Dashboard\anomaly_injector.py:228: Deprecatio
nWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="anomaly-injector", **MQTT_CLIENT_ARGS)
[CONNECTED] localhost:8883
[TOPIC]      hydroficient/grandmarina/device-002/sensors
[SENDING]    Anomalies every 3 seconds (Ctrl+C to stop)

[ANOMALY] High Pressure
Seq: 50001 | Pressure: 68.38 PSI | Flow: 48.31 LPM | Gate: 43.58%
HMAC: VALID | Timestamp: FRESH | Sequence: NEW
(All rule checks will pass - only AI should flag this)

[ANOMALY] Low Pressure
Seq: 50002 | Pressure: 52.63 PSI | Flow: 53.62 LPM | Gate: 44.68%
HMAC: VALID | Timestamp: FRESH | Sequence: NEW
(All rule checks will pass - only AI should flag this)

[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 46.66 LPM | Seq: 9
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: Normal (score: 0.006)
Stats: 12 accepted, 0 rejected, 4 AI anomalies (12 total)

[AI FLAGGED] Device: HYDROLOGIC-Device-002 | Flow: 57.22 LPM | Seq: 50004
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: ANOMALY (score: -0.004)
Stats: 13 accepted, 0 rejected, 5 AI anomalies (13 total)

[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 53.39 LPM | Seq: 10
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: Normal (score: 0.106)
Stats: 14 accepted, 0 rejected, 5 AI anomalies (14 total)

[AI FLAGGED] Device: HYDROLOGIC-Device-002 | Flow: 39.44 LPM | Seq: 50005
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: ANOMALY (score: -0.077)
Stats: 15 accepted, 0 rejected, 6 AI anomalies (15 total)
```

EAVESDROP + SEQUENCE + HMAC ATTACK + ANOMALY INJECTION: With all of our security layers including an AI model we are able to see good results as hmac and sequence attacks are blocked as well.

```
[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 50.18 LPM | Seq: 80
Failed check: SEQUENCE | Reason: Sequence 80 <= last seen 82
Stats: 27 accepted, 2 rejected, 17 AI anomalies (29 total)

[ACCEPTED] Device: HYDROLOGIC-Device-002 | Flow: 51.0 LPM | Seq: 50018
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: Normal (score: 0.144)
Stats: 28 accepted, 2 rejected, 17 AI anomalies (30 total)

[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 53.49 LPM | Seq: 83
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: Normal (score: 0.098)
Stats: 29 accepted, 2 rejected, 17 AI anomalies (31 total)

[AI FLAGGED] Device: HYDROLOGIC-Device-002 | Flow: 62.94 LPM | Seq: 50019
HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS | AI: ANOMALY (score: -0.123)
Stats: 30 accepted, 2 rejected, 18 AI anomalies (32 total)
```


- **Simulated realistic cyberattacks (spoofing, replay, and MitM) on MQTT-based infrastructure to identify and remediate vulnerabilities in high-frequency industrial data streams.**
- **Implemented secure device-to-cloud communication using TLS/mTLS and Mosquitto ACLs, evaluating security overhead by documenting an 87ms increase in handshake latency.**
- **Developed a real-time anomaly detection prototype using Python, Streamlit, and an Isolation Forest model; achieved a Precision of 73.6%, Recall of 70%, and an F1-score of 75.7% with a model training latency of 0.389 seconds.**
- **Authored structured security reviews, runbooks, and incident response reports, delivering actionable risk mitigation strategies and technical documentation for IoT stakeholders.**